

The rollout leaf: judging a position by playing it out

2026-08-03, extended overnight into 08-04. Written after MEDICHAM went from 89.2% to 98.5% click coverage in one session, which is the only reason this is now buildable. Will's question started it: *"cant it just be all the same calcs we already do"*.

1. The claim, and the number it rests on

Every search in `docs/LOOKAHEAD-design.md` maximises a leaf. That leaf is PORYGON2, and `data/porygon2c.json` scores it against its own baselines:

coin	50.38%	
material sign	60.28%	<- "who has more bodies and HP"
PORYGON2 (k=200)	63.70%	

The entire learned value function is worth 3.4 accuracy points over counting bodies. A search that maximises it is, to within 3.4 points, a search for "which move removes the most material next turn" — which is what a greedy damage policy already does. That is the strongest available explanation for why one step of foresight measures small once the clock artifact is removed (`+2.29` , not `+4.91` — `engine/lookahead_clock_control.py`).

So the leaf is the constraint, not the search. This document is about replacing it.

The replacement: seed MEDICHAM from the real position and play it out. The answer is then a measured win rate rather than a snapshot judgement, and it is produced by the same doubles engine that already ships in ALAKAZAM's Future Sight.

2. Why this is buildable today and was not yesterday

`battleInit` never reset HP — it takes the Pokemon objects as handed over. MEDICHAM could always have been seeded mid-game; nothing ever did it because `winProb2` builds fresh mons from names.

What was genuinely missing was coverage. On 2026-08-03 the engine turned an unmodelled click into a **no-op turn**, which is not neutral noise: it biases against exactly the utility and multi-turn moves that decide games. `engine/medicham_coverage.js` measured it and the session closed the gap:

start of session	89.2%
Trick Room	+1.18
heal family (Roost, Recover, Life Dew)	+0.80
redirection (Rage Powder, Follow Me)	+1.78
screens (Reflect, Light Screen, Aurora Veil)	+1.69
switching — voluntary, pivot, faint	+2.12
weather setters	+0.26
Helping Hand	+0.67
ally boosts, Perish Song, Yawn	+0.52
Leech Seed (routed), Super Fang	+0.30

now	98.5%
-----	-------

Plus the corrections Will supplied while it was being built: Aurora Veil needs snow; Brick Break, Psychic Fangs and Raging Bull break screens; Light Clay extends them; Prankster's +1 (which was modelled *nowhere*, so a Prankster screen went up *after* the attack it was meant to blunt).

Still missing, and stated rather than buried: Parting Shot's Atk/SpA drop (`statChangeInCode` , and no artifact this engine reads carries the numbers), the 22 `lowersTarget` moves whose boost table lives on the dex object this bundle cannot reach, and the 1.5% tail that remains no-ops.

3. The literature, including the result that argues against this design

3.1 Truncated rollouts are the right shape, and MEDICHAM already has one

Tesauro's rollout work established that trials need not be played to completion: stop after a few steps and substitute an estimate of the position reached. That costs less CPU **and has lower variance per trial**, because a real-valued estimate replaces an integer outcome reached through many random steps ([Monte Carlo Rollouts in RL](#); [Tesauro & Galperin, On-line Policy Improvement using Monte-Carlo Search](#)). Full Monte Carlo is low bias and high variance; bootstrapping trades variance for bias; truncation is the compromise, and an intermediate λ beats both endpoints ([bias-variance in RL](#)).

`battleOver` already stops at `S.turn>=20` and `battleResult` already returns a graded readout — bodies first, then summed HP fraction, 0.5 for a dead tie. That is a truncated rollout with a material estimate at the horizon, and it was there before this document.

3.2 A STRONGER rollout policy does not reliably make the search stronger

The most useful counter-intuitive result in the area: *the strength of a rollout policy as a standalone player is not a good predictor of its strength inside a Monte-Carlo search*. Objectively stronger rollouts frequently make UCT **worse**, and heavy rollouts help only when they avoid becoming low-variance ([An Analysis of Monte Carlo Tree Search](#); [MCTS-Minimax hybrids](#)).

This is a live risk here and it is cheap to get wrong: the obvious "improvement" after this ships is to make `chooseAction` play better inside the rollout. **That may reduce accuracy, and it must be measured rather than assumed.** It also means MAG is not automatically the right rollout policy just because it is our best player.

3.3 The strongest comparable bot decided rollouts were NOT worth it

Foul Play is a competitive Showdown bot on a custom Rust engine (`poke-engine`). It reaches **Top 100 in Gen9 OU (80% GXE)**, **#1 in Gen9 RandomBattleBlitz (84%)** and **71–88% GXE across formats** — and it runs MCTS "**guided by a custom evaluation function rather than rollouts.**" It also abandoned expectiminimax because search beyond about five turns timed out, and it groups the 32 damage rolls by whether they cause a faint rather than branching on each.

BUT IT PLAYS SINGLES, AND WE PLAY DOUBLES. Every format quoted — Gen9 OU, Gen4 RandomBattle, Gen9 RandomBattleBlitz — is 1v1. Will asked and the answer changes how much the result transfers:

- A singles side picks ONE action from ~9. A doubles side picks a PAIR, so the joint action space is that squared, and the matrix is squared again across both sides. `engine/truncation_curve.js` measured a median of 8 candidates per slot in our corpus, so unpruned is ~64 joint actions per side against Foul Play's ~9.

- "Depth 10 or more" is therefore a much cheaper claim in their game than in ours. Their own expectiminimax gave out past ~5 turns *in singles*.
- Redirection, spread damage, Wide Guard, Follow Me and partner interactions do not exist in singles at all, and those are precisely the mechanics a static evaluation function struggles to score and a playout gets for free.

So it is real evidence that a static eval can carry a strong bot, and weak evidence about what wins in doubles. Recorded rather than in a footnote. Two honest readings:

- **Against us:** a good static eval plus deep DUCT search beats shallow rollouts, and our effort is better spent on the eval and the search than on the playout.
- **For us:** Foul Play *has* a good custom evaluation function. We do not — ours is worth 3.4 points over counting bodies. Rollouts are the cheapest way to get a better leaf *when you do not already have one*, which is precisely our position.

Both readings agree on the measurement in §5, which is why that is the gate.

3.4 Simultaneous moves, unchanged from LOOKAHEAD-design §3.2

Both players commit blind, so the root is a matrix game. DUCT is what Foul Play uses and what the literature recommends; regret matching converges and `engine/slowking/nash.py:solve_rm` already implements it.

None of that is in scope here. This document is only about the leaf.

3.5 The field's own warning still stands

`docs/LITERATURE-v2.md` §2: Metamon (arXiv 2504.04395) reaches top-10% of human players with offline RL and **no search at all**. The [PokéAgent Challenge](#) (22M+ trajectories) finds specialised RL and search still beat generalist LLMs. Neither route needs a rollout leaf.

4. The design

4.1 One function

```
rolloutWinProb(board, side, N) -> P(side wins)
```

Seed. Every Pokemon on both sides — active and benched — becomes a MEDICHAM mon via the existing `mc_key.js` resolver and `buildMon`, then has its live state written onto it: current HP, status, stat stages, item. The bench matters now that switching exists; before this session it would have been decorative.

Do not re-apply entry effects. `battleInit` runs `applyEntryEffects` and Intimidate on the actives, which is correct when a battle starts and wrong for a mid-game seed — the Intimidate already happened in the real game and applying it again would drop the foe's Attack a second time. A seeded init skips it.

Roll out. `battleTurn` to `battleOver`, N times, count wins. `chooseAction` is the playout policy on both sides.

Report. Wins / N. And, because a rollout estimate without an interval invites reading noise as signal (the same argument `champions_sim.winProb` already makes), a Wilson interval on the same call.

4.2 What N has to be

A rollout is a Bernoulli sample, so the standard error at $p \approx 0.5$ is $0.5/\sqrt{N}$. N=20 gives ± 11 points, N=100 gives ± 5 , N=400 gives ± 2.5 . The leaf being replaced is 63.7% accurate, so an estimator whose own noise is ± 11 points is not obviously an improvement. **N is a measured parameter, not a guess**, and §5 measures accuracy as a function of it.

MEASURED 2026-08-03, and it contradicts the paragraph above. `engine/rollout_r1.js` with `N_LIST=10,40,160` scores every budget on the SAME 4,487 positions in one replay, so the comparison is paired and cannot be confounded with which positions each budget happened to see:

ROLLOUT n=10	64.03%	Brier 0.2739
ROLLOUT n=40	64.68%	Brier 0.2657
ROLLOUT n=160	64.48%	Brier 0.2645

Accuracy is FLAT in N. Sixteen times the compute moves it by less than half a point, and not monotonically. Brier improves a little — that is the variance reduction arriving where it should, in the calibration rather than in the verdict — but the gate does not move.

So the leaf is **bias-limited, not variance-limited**, and this settles two things at once:

- **"Run more rollouts" is dead as a lever.** Whatever is wrong is wrong in the engine or in `chooseAction`, and more samples of a biased estimator converge on the bias.
- **The leaf is cheap.** $n=10$ is as accurate as $n=160$, so the per-leaf cost budget in §4.3 is an order of magnitude smaller than assumed. If R1 ever passes, affordability is not the obstacle.

It also sharpens §3.2's warning into the live question: the payout policy is the suspect, and the literature says making it *stronger* is not reliably the fix.

4.2.1 The bias was the payout, and the fix is MORE randomness

`chooseAction` is deterministic greedy. Every payout from one position replays the same line, so N samples agree with **each other** rather than with reality. The calibration table shows the result — the estimator saturates:

rollout says	n	actually wins
0-10%	2,245	26.0%
90-100%	2,612	75.9%

53% of all positions land in those two bins, and they are wrong by 22-29 points.

§3.2's warning applied exactly: *heavy rollouts help only when they avoid becoming low-variance*. Swept on the same 1,456 positions at $n=40$, with `explore` the probability that a Pokemon clicks a random legal move instead of the greedy pick:

<code>explore=0</code>	64.42%	Brier 0.2647	logloss 1.7400
<code>explore=0.15</code>	64.42%	0.2509	1.4309
<code>explore=0.35</code>	65.11%	0.2391	1.0691
<code>explore=0.6</code>	65.73%	0.2265	0.9293
<code>explore=0.8</code>	66.62%	0.2190	0.9119
<code>explore=1.0</code>	68.13%	0.2192	0.8849

Monotone in every column, and a FULLY RANDOM payout wins. +3.71 accuracy points over greedy, and +3.29 over material — against PORYGON2's published +3.42 over the same baseline.

And N starts behaving: at `explore=1`, more samples buy **calibration** rather than accuracy (logloss 1.396 -> 0.885 -> 0.764 at $n=10/40/160$). That is the right currency for a leaf a search will average rather than threshold.

The randomness lives in the rollout harness, **not** in `chooseAction` — that is the Tower's policy and the live bot's, and randomising it would change shipped behaviour to fix a leaf. `explore=0` reproduces the old payout exactly, so every sweep contains its own incumbent.

4.3 Cost, against the budget already established

`data/lookahead-cost.json` puts a Showdown fork at ~4.6 ms. A MEDICHAM turn is far cheaper — no protocol, no serialization — but a rollout is up to 20 of them, and a leaf costs N rollouts. Cost per leaf is therefore $N \times 20 \times (\text{cost of a MEDICHAM turn})$ and must be measured before any search quotes a matrix size. This is the same discipline as G2, and G2's first cost figure was wrong because the instrument was broken, so it is measured here on many boards rather than one.

5. Gates

R1 — NOT ESTABLISHED. The published PASS has no reproducible evidence. Artifact:

`data/rollout-r1.json`, written by `engine/rollout_r1_artifact.js`.

What was published here until 2026-08-04, and is now withdrawn: *"R1 — PASSED ON THE BASELINE. 9,201 positions, fully random payout: rollout 68.18% against material's 65.26%, +2.91 points, 95% CI 1.79 to 4.04."* That sentence is kept verbatim because a prior conclusion is never silently rewritten. It is not retracted as *wrong* — it is retracted as **uncheckable**.

`engine/rollout_r1.js` computed it with `console.log` and wrote no artifact. The only file it wrote is the row dump `data/rollout-r1-rows.jsonl`, and recomputing the gate from that dump — same formulas, `engine/rollout_r1.js :225-272` — gives a different answer:

judge	accuracy	Brier	log-loss
coin (= the base rate)	52.46%		
material: porygon2 form (graded)	65.26%	0.2127	0.6124
ROLLOUT (the dumped column)	65.72%	0.2573	1.7674

rollout minus material +0.46 points 95% CI -0.72 to +1.63
discordant: rollout-only-right 1,538, material-only-right 1,496, of 9,201
-> UNDECIDED. The interval spans zero.

The material column reproduces the published 65.26% **exactly**, so it is the same 9,201 positions. The rollout column is not the same rollout. Its reliability bins reproduce §4.2.1's **greedy** saturation table count-for-count — 2,245 at 26.0% and 2,612 at 75.9% — so the surviving dump is the `explore=0` incumbent, not the fully random payout the verdict was computed from. `mpy` is deterministic given a position, so a matching material accuracy proves the same **sample**, never the same **run**. The artifact asserts that comparison against this document rather than asserting it in prose.

The dump records neither `N` nor `explore` nor any build digest, which is why the two runs cannot be told apart by any field in the file. `engine/rollout_r1.js` now writes `data/rollout-r1-rows.meta.json` beside every dump carrying both, plus content digests of every source the leaf reaches. The committed rows predate it.

What R1 currently supports: the greedy payout is not separated from counting bodies on this sample (+0.46, CI spans zero, and the split-half spread of the run itself ranges 0.43 to 2.01 points — larger than the effect). **What it does not support:** the 68.18% figure, or anything resting on it, including `--rollout-explore` defaulting to `1.0`. Re-run `engine/rollout_r1.js` with `EXPLORE_LIST=1` and `DUMP` set, then `node engine/rollout_r1_artifact.js`.

The cross-language head-to-head against PORYGON2 itself was built and **withdrawn** — see `engine/rollout_r1_join.py` , which fails its own alignment check and says so. Its numbers are preserved at `data/rollout-r1-withdrawn-join.json` with `withdrawn: true` . It held the name `data/rollout-r1.json` until 2026-08-04, and `engine/status.js` duly printed a withdrawn result as this gate's status; `status.js` now refuses to print any artifact carrying `withdrawn: true` .

R1, as originally specified — the rollout is a better JUDGE than PORYGON2. No search, no matrix, no equilibrium. Take clean human games with known winners, seed MEDICHAM at each turn, roll out, and ask whether the win rate predicts the actual winner better than the k-NN does. Same positions, same labels, and the material baseline recomputed on the same sample so the comparison is not against a published number from a different set.

if rollout accuracy <= 63.7% -> the leaf does not improve, and the whole idea dies for the price of an afternoon. This is the cheap null and it is the reason R1 exists before anything else. if rollout accuracy > 63.7% -> the leaf improves, and every search built on it inherits that.

R2 — it is affordable. PASSED. `engine/rollout_r2.js` , timed on 200 games of real boards rather than one:

n=10	5.83 ms median	10.62 p90	21.62 max	
n=40	22.39			
n=160	89.86			
K=3, 81 cells	0.47 s median	1.75 s worst	affordable	
K=4, 256 cells	1.49 s	5.53 s	MARGINAL	

A Showdown fork costs 4.52 ms and buys ONE simulated turn. A rollout leaf costs 5.83 ms and plays the position out, up to 20 turns, ten times. Roughly 200x the simulated turns per millisecond. Whatever kills this design, it will not be the cost of the leaf.

R3 — the pick changes. PASSED, and only because it was made to prove its own noise floor. `engine/rollout_r3.js` . At each real decision, force each of my top-K joint actions, let the opponent play its own policy, run one turn, roll the rest out, take the argmax; compare against MAG's own ranking of the same candidates.

The first run said **67.4% divergence**, which is a headline. It is not a result:

N=20	divergence 67.4%	the same search disagreeing with ITSELF	71.7%
N=60	divergence 77.3%		50.0%
N=200	divergence 81.8%		45.5%
N=200, 185 decisions	72.4%		43.8%

An argmax over K² noisy estimates disagrees with anything, so the control is the same search on a different seed — where the truth is 0.00 by construction. **At N=20 the floor was HIGHER than the divergence.** Without it, "the search picks differently two-thirds of the time" would have been noise reported as a finding.

At N=200 the floor falls to ~44% while divergence rises to ~72%, so the gap is real and these are different players. **But 44% is not a footnote:** a search that disagrees with itself on nearly half its decisions is not yet a stable player, and that will make R4's SPRT noisy. Raising N is the obvious lever and it is cheap — R2 says a leaf is 5.83 ms at n=10.

R4 — it wins. SPRT against the shipped greedy player, read as it goes.

6. What would make this fail, written down first

- **Foul Play chose the other branch** (§3.3), with a strong ladder record behind the choice.
 - **A stronger payout is not reliably better** (§3.2), so the obvious follow-up work may be negative.
 - **3.3% of clicks are still no-ops** and the bias is one-directional: unmodelled utility moves look worthless. Parting Shot's debuff is missing while its switch is modelled — half a move, and the half that decides where it is played.
 - `chooseAction` **is not a strong player**. A rollout inherits whatever its policy is, and if that policy never switches voluntarily, positions whose answer is a switch are still misjudged.
 - **Variance**. At small N the leaf is noisier than the model it replaces (§4.2).
 - **MEDICHAM is not Showdown**. ADR-001 pins Showdown as the real engine for a reason; every rollout is a simplification, and `data/damage-validation.json` bounds how good the damage half is.
-

7. What this does not claim

It does not claim the search will win, or that rollouts beat a good static evaluation — the best documented bot in this game says otherwise. It claims one thing, and R1 measures it: **our leaf is worth 3.4 points over counting bodies, and a rollout through an engine that now represents 96.7% of real clicks may be worth more.**